



Audity

Self-Hosted Docker Edition

Technical Concept

Version	V4.Final
Status	Technical Concept
Deployment	Self-Hosted Docker / Single-Tenant Stack
Product	Audity — Streamline the Audit Process
Year	2026

This document describes the full technical concept for the Audity Self-Hosted Docker Edition — a security-first, local-first audit and GRC assessment platform for CISOs, security consultants, auditors, and GRC professionals.

Table of Contents

Document Purpose

1. Executive Summary

2. Core Principles

- 2.1 Security First
- 2.2 Local / Self-Hosted First
- 2.3 Tenant Isolation by Deployment
- 2.4 Human-in-the-Loop Assessment
- 2.5 Traceability by Design

3. Target Users

4. Deployment Model

- 4.1 Recommended Deployment Pattern
- 4.2 Single-Tenant Stack

5. High-Level Architecture

6. Container Structure

- 6.1 audity-web
- 6.2 audity-api
- 6.3 audity-worker
- 6.4 audity-db
- 6.5 audity-storage
- 6.6 audity-redis
- 6.7 audity-ai (optional future component)

7. Minimum Hardware Requirements

- 7.1 Minimum Test / Lab Deployment
- 7.2 Recommended Production (≤ 25 users)
- 7.3 Comfortable Production
- 7.4 Storage Planning
- 7.5 Multi-Tenant Server Consolidation

8. Security Architecture

- 8.1 Password Security
- 8.2 Multi-Factor Authentication
- 8.3 Session Security
- 8.4 Customer Data Encryption
- 8.5 Encryption Key Handling
- 8.6 Evidence File Security
- 8.7 Authorization and RBAC

9. Immutable User Activity Logging

- 9.1 Purpose
- 9.2 What Must Be Logged
- 9.3 Click-Level Logging
- 9.4 Before / After Change Tracking
- 9.5 Tamper-Resistant Design
- 9.6 Activity Log Access Control
- 9.7 Activity Log UI

10. Audit Logging

11. Application Modules

- 11.1 Guided Assessment Workflow
- 11.2 Customer / Client Management

- 11.3 Assessment Setup
- 11.4 Scope & Context
- 11.5 Framework Library / Compliance Engine
 - 11.5.1 Proprietary Framework Handling
 - 11.5.2 ISO 27001 Support Model
 - 11.5.3 Bring Your Own Framework Content
 - 11.5.4 Framework License Metadata
 - 11.5.5 Framework Library UI
 - 11.5.6 Open and Public Framework Strategy
 - 11.5.7 Required Disclaimer
- 11.6 Guided Questions
- 11.7 Finding Review
- 11.8 Risk Register
- 11.9 Roadmap Builder

12. Branded Report Builder

- 12.1 Purpose
- 12.2 Admin-Controlled Branding
- 12.3 Modular Report Design System
- 12.4 Required Author Information
- 12.5 Report Export Formats
- 12.6 Report Integrity

13. Secure Email Module

- 13.1 Purpose
- 13.2 Supported Sending Modes
- 13.3 Encrypted Report Package
- 13.4 Email Configuration
- 13.5 Recipient Controls
- 13.6 Email Audit Trail
- 13.7 Admin Controls

14. Import / Export

15. Database Design Overview

16. Docker Compose Concept

17. Environment Variables

18. Backup and Recovery

19. Logging and Monitoring

20. Security Hardening Recommendations

21. Future AI Architecture

22. Open Source Positioning

23. Implementation Phases

- Phase 1 — Docker Foundation
- Phase 2 — Authentication and Security
- Phase 3 — Data Persistence
- Phase 4 — Traceability and Integrity
- Phase 5 — Reporting and Branding
- Phase 6 — Secure Email Module
- Phase 7 — Evidence and Reporting Hardening
- Phase 8 — Platform Hardening
- Phase 9 — Optional AI

24. Alpha Limitations

25. Final Target State

Document Purpose

This document describes the technical concept for a self-hosted Docker-based edition of **Audity**.

Audity is designed as a security-first, local-first and self-hostable audit and security assessment platform for CISOs, security consultants, auditors, and GRC professionals.

The goal of the Docker edition is to provide a free and open-source deployment model that allows organizations to run Audity inside their own controlled infrastructure without relying on a hosted SaaS platform.

The self-hosted edition is intended for organizations that want full control over:

- customer data
- assessment data
- evidence files
- findings
- risk registers
- roadmap data
- report exports
- audit logs
- user activity logs
- authentication and MFA
- encryption keys
- infrastructure and backup strategy

1. Executive Summary

Audity helps security and GRC professionals move faster from assessment planning to control review, findings, risk register, remediation roadmap, and final report.

The Docker edition allows every organization or customer to run a dedicated Audity stack with its own database, storage, secrets, authentication configuration, encryption scope, audit logs, user activity logs, and optional future AI components.

Audity does not replace professional judgment. It provides structure, traceability, and repeatability for audit and security assessment workflows.

2. Core Principles

2.1 Security First

Audity processes highly sensitive data, including:

- customer infrastructure details
- audit evidence
- security weaknesses
- compliance gaps
- control maturity ratings
- findings
- risk ratings
- remediation plans
- management reports
- framework mappings

- privileged access issues
- incident response maturity gaps
- user activity history
- assessment decision history

The platform must treat this data as confidential, integrity-sensitive, and audit-relevant by default.

2.2 Local / Self-Hosted First

The Docker edition is designed to run inside an organization's own infrastructure.

Supported deployment targets:

- local server
- internal VM
- private cloud VM
- customer-managed Kubernetes cluster
- private lab environment
- consultant-controlled assessment environment
- isolated managed tenant stack

No hosted Audity backend is required.

2.3 Tenant Isolation by Deployment

The preferred model is one Audity stack per organization or customer.

```
TEXT
Customer A → Audity Stack A
Customer B → Audity Stack B
Customer C → Audity Stack C
```

Each stack has its own:

- database
- object storage / evidence storage
- secrets
- encryption keys
- audit logs
- user activity logs
- report exports
- optional future AI runtime

This reduces the risk of cross-customer data leakage compared to a shared multi-tenant database model.

2.4 Human-in-the-Loop Assessment

Audity may suggest findings, risks, and roadmap items based on rules and framework mappings, but the user remains responsible for final assessment decisions.

The application must not claim that an organization is:

- certified
- fully compliant
- guaranteed compliant
- audit-proof

unless this conclusion is explicitly made by a qualified professional outside the automated suggestion engine.

2.5 Traceability by Design

Every relevant user action must be traceable.

Audity must make it technically difficult for users to silently manipulate assessment results, answers, findings, risk ratings, evidence status, report content, or roadmap data after the fact.

All relevant actions must be recorded in an append-only, tenant-admin-visible activity log.

3. Target Users

Audity Self-Hosted Docker Edition is intended for:

- CISOs
- security consultants
- GRC professionals
- auditors
- internal security teams
- SOC / detection leads
- IT security managers
- compliance teams
- managed security service providers
- organizations that do not want to upload audit data to external SaaS platforms

4. Deployment Model

4.1 Recommended Deployment Pattern

The recommended self-hosted deployment is a Docker Compose stack.

```
TEXT
Audity Tenant Stack
■■■ audity-web
■■■ audity-api
■■■ audity-worker
■■■ audity-db
■■■ audity-storage
■■■ audity-redis
■■■ audity-reverse-proxy optional
■■■ audity-ai optional future component
```

4.2 Single-Tenant Stack

Each customer or organization should run a separate stack.

Example:

```
TEXT
audity-artemis/
■■■ docker-compose.yml
■■■ .env
■■■ volumes/
■ ■■■ postgres/
■ ■■■ storage/
■ ■■■ backups/
■■■ config/
```

A second customer gets a separate stack:

```
TEXT
audity-nordbank/
■■■ docker-compose.yml
■■■ .env
■■■ volumes/
■   ■■■ postgres/
■   ■■■ storage/
■   ■■■ backups/
■■■ config/
```

This model creates operational separation between customers.

5. High-Level Architecture

```
TEXT
Browser
  ↓ HTTPS
Reverse Proxy / TLS Termination
  ↓
Audity Web Frontend
  ↓
Audity API Backend
  ↓
Authentication / Authorization Layer
  ↓
Application Services
  ■■■ Assessment Service
  ■■■ Framework Engine
  ■■■ Finding Service
  ■■■ Risk Register Service
  ■■■ Roadmap Service
  ■■■ Report Service
  ■■■ Evidence Service
  ■■■ User Management Service
  ■■■ Email Delivery Service
  ■■■ Audit Log Service
  ■■■ User Activity Log Service
  ↓
PostgreSQL Database
Object Storage / Evidence Storage
Redis / Queue
Backup Storage
```

6. Container Structure

6.1 audity-web

Frontend container serving the web application.

Responsibilities:

- user interface
- guided assessment workflow
- dashboard views
- framework library UI
- report preview UI
- settings UI

- admin branding settings UI
- report design builder UI

Recommended stack:

- React
- TypeScript
- Vite
- Tailwind CSS or equivalent CSS system
- static production build served by NGINX or Caddy

6.2 audity-api

Backend API container.

Responsibilities:

- authentication
- authorization
- session handling
- MFA verification
- tenant configuration
- assessment persistence
- framework engine logic
- evidence metadata handling
- findings, risks, roadmap persistence
- report generation orchestration
- email module integration
- audit logging
- immutable user activity logging

Recommended stack:

```
TEXT
Node.js + TypeScript + Fastify/NestJS
```

6.3 audity-worker

Background worker container.

Responsibilities:

- PDF generation
- branded report rendering
- encrypted report package generation
- email delivery jobs
- report export jobs
- backup jobs
- async evidence processing
- future scheduled tasks
- future AI jobs

6.4 audity-db

Database container.

Recommended database:

TEXT
PostgreSQL

Responsibilities:

- users
- roles
- assessments
- framework mappings
- findings
- risks
- roadmap items
- reports
- audit logs
- user activity logs
- application settings
- report branding configuration
- email delivery records

6.5 audity-storage

Evidence and file storage component.

Options:

- local Docker volume
- MinIO container
- S3-compatible object storage
- mounted encrypted filesystem

Recommended for self-hosted Docker:

TEXT
MinIO or local mounted volume

Storage is used for:

- evidence files
- generated reports
- encrypted report packages
- exported project archives
- attachments
- branding assets such as company logos
- future AI document stores

6.6 audity-redis

Optional Redis container.

Used for:

- job queues
- rate limiting
- temporary workflow state
- cache
- background task coordination

6.7 audity-ai optional future component

Optional tenant-local AI runtime.

Possible future components:

- Ollama
- local LLM inference container
- tenant-specific vector database
- tenant-specific embedding store

Important rule:

```
TEXT
AI data must never be shared across customer stacks.
```

7. Minimum Hardware Requirements per 25-User Tenant

The following sizing assumes one self-hosted Audity stack for one tenant with up to 25 registered users and no AI component.

Audity is not expected to be CPU-heavy without AI. The main resource drivers are PostgreSQL, report generation, evidence storage, concurrent users, and backup retention.

7.1 Minimum Test / Lab Deployment

Suitable for internal testing, lab use, and small non-production evaluations.

```
TEXT
CPU:      2 vCPU
RAM:      4 GB
Storage:  40-60 GB SSD
OS:       Linux server
Runtime:  Docker + Docker Compose
```

Limitations:

- not recommended for production
- limited report generation headroom
- limited evidence storage
- no comfortable operational reserve

7.2 Recommended Production Deployment per 25-User Tenant

Suitable for a small production tenant without AI.

```
TEXT
CPU:      4 vCPU
RAM:      8 GB
Storage:  100-200 GB SSD/NVMe
OS:       Linux server
Runtime:  Docker + Docker Compose
```

Recommended for:

- up to 25 users
- several concurrent assessments
- regular report generation
- moderate evidence storage
- daily backups
- audit and activity logging

7.3 Comfortable Production Deployment per 25-User Tenant

Recommended when the tenant stores evidence files, generates larger reports, or needs more operational headroom.

```
TEXT
CPU:      4-8 vCPU
RAM:      16 GB
Storage:  250-500 GB NVMe
OS:       Linux server
Runtime:  Docker + Docker Compose
```

Recommended for:

- active consulting teams
- frequent report generation
- larger evidence files
- longer backup retention
- heavier audit log volume

7.4 Storage Planning

Storage must account for:

- database
- evidence files
- generated PDF reports
- encrypted report packages
- exported project archives
- audit logs
- user activity logs
- backups

Recommended initial storage allocation:

```
TEXT
Small tenant:      100 GB
Active tenant:     250 GB
Evidence-heavy:    500 GB+
```

7.5 Multi-Tenant Server Consolidation

If multiple tenant stacks run on one larger server, CPU can be overcommitted more safely than RAM and storage.

General rule:

```
TEXT
CPU:      can be overcommitted within reason
RAM:      should not be aggressively overcommitted
Storage:  must be planned conservatively
Backups:  must be calculated separately from live data
```

For production, each tenant stack should have Docker resource limits.

8. Security Architecture

8.1 Password Security

Passwords must not be encrypted reversibly.

Passwords must be hashed using a dedicated password hashing algorithm.

Recommended:

```
TEXT
Argon2id
unique salt per password
optional pepper stored outside the database
```

Alternative acceptable options:

- bcrypt
- PBKDF2 with strong parameters

Requirements:

- never store plaintext passwords
- never log passwords
- never send passwords back to the client
- password reset tokens must be single-use and time-limited
- password reset tokens must be hashed in the database

8.2 Multi-Factor Authentication

Audity must support MFA through authenticator apps.

Supported MFA method for the first self-hosted release:

```
TEXT
TOTP
```

Compatible apps include:

- 1Password
- Microsoft Authenticator
- Duo Mobile
- Google Authenticator
- Authy
- Bitwarden Authenticator
- Aegis Authenticator

MFA requirements:

- TOTP enrollment via QR code
- recovery codes
- MFA reset flow for administrators
- optional tenant-wide MFA enforcement
- audit log entry when MFA is enabled, disabled, reset, or bypassed

8.3 Session Security

Sessions must be protected with:

- secure HTTP-only cookies
- SameSite cookie policy
- CSRF protection if cookie-based auth is used
- session expiration
- idle timeout

- refresh token rotation if token-based auth is used
- logout from all devices
- session revocation by admin

8.4 Customer Data Encryption

Audity must encrypt sensitive customer data at rest.

Recommended model:

```
TEXT
Database-level encryption
+
Application-level field encryption for highly sensitive data
+
Encrypted storage volume or object storage encryption
```

Sensitive fields should include:

- assessment notes
- answers to questions
- findings
- risks
- business impact descriptions
- evidence notes
- report drafts
- customer infrastructure details
- uploaded evidence metadata
- API secrets if ever introduced

8.5 Encryption Key Handling

Recommended self-hosted model:

- application master key configured through environment variable or secret file
- per-project or per-tenant data encryption keys derived or stored encrypted
- key rotation plan for future versions
- encryption keys must not be committed to Git
- encryption keys must not be stored inside the container image

Example environment variables:

```
ENV
AUDITY_APP_SECRET=change-me
AUDITY_ENCRYPTION_KEY=base64-encoded-key
AUDITY_DATABASE_URL=postgres://audity:password@audity-db:5432/audity
```

8.6 Evidence File Security

Evidence uploads must be treated as sensitive.

Requirements:

- file size limits
- file type restrictions
- malware scanning optional but recommended
- no public object storage buckets
- short-lived signed download URLs if object storage is used
- access checks on every file download

- audit log for upload, download, delete, and export actions
- encrypted storage volume or object storage encryption

8.7 Authorization and RBAC

Audity must include role-based access control.

Recommended roles:

```
TEXT
Instance Admin
Tenant Admin
Assessment Manager
Auditor
Contributor
Reviewer
Viewer
```

Example permissions:

```
TEXT
users.invite
users.disable
roles.manage
assessment.create
assessment.edit
assessment.delete
assessment.view
finding.create
finding.approve
risk.edit
risk.accept
roadmap.edit
report.export
evidence.upload
evidence.download
auditlog.view
activitylog.view
settings.manage
branding.manage
email.manage
```

Every API request must verify:

```
TEXT
Who is the user?
Which instance or tenant do they belong to?
Which role do they have?
Which resource are they trying to access?
Are they allowed to perform this action?
```

9. Immutable User Activity Logging

9.1 Purpose

Audity must include a detailed user activity logging system for every tenant.

The purpose is to ensure traceability and prevent users from silently changing assessment results, answers, findings, risk ratings, evidence status, or report content after the fact.

The activity log must be visible only to authorized Tenant Admins and Instance Admins.

Standard users must not be able to edit, delete, hide, or manipulate their own activity history.

9.2 What Must Be Logged

The activity log should record all relevant user interactions, including:

- login and logout
- failed login attempts
- MFA events
- page navigation
- opening an assessment
- opening a customer project
- answering a question
- changing an answer
- changing a maturity score
- changing evidence status
- changing confidence level
- editing notes
- creating a finding
- editing a finding
- deleting or dismissing a suggested finding
- changing finding priority
- changing likelihood or impact
- accepting a risk
- changing a risk rating
- changing treatment option
- changing owner or due date
- moving roadmap items
- exporting a report
- sending a report by email
- exporting project backup
- importing project backup
- uploading evidence
- downloading evidence
- deleting evidence
- changing report branding
- changing tenant settings
- changing user roles

9.3 Click-Level Logging

Audity should support click-level user activity logging for sensitive workflows.

Examples:

```
TEXT
User clicked "Save Answer"
User clicked "Accept Finding"
User clicked "Dismiss Finding"
User clicked "Export Report"
User clicked "Send Encrypted Email"
User opened "Risk Register"
```

For high-volume UI actions, the system may group non-critical navigation events to reduce noise, but all assessment-changing actions must be logged individually.

9.4 Before / After Change Tracking

For every data-changing event, the log must capture before and after values.

Example:

```
JSON
{
  "action": "control_answer.updated",
  "field": "maturityScore",
  "before": 1,
  "after": 3,
  "userId": "usr_123",
  "assessmentId": "assess_456",
  "timestamp": "2026-05-15T12:00:00Z"
}
```

This is required for:

- answer changes
- score changes
- evidence status changes
- finding edits
- risk edits
- report content edits
- ownership changes
- due date changes
- roadmap changes

9.5 Tamper-Resistant Design

The user activity log must be append-only.

Requirements:

- no update or delete operation through the application layer
- only append new events
- restricted database permissions for the application user
- optional hash chaining for tamper evidence
- optional periodic log export
- optional external log forwarding in future versions

Recommended event integrity model:

```
TEXT
Event N contains hash of Event N-1
Event hash = hash(timestamp + actor + action + entity + payload + previousHash)
```

This creates a tamper-evident chain.

9.6 Activity Log Access Control

Only the following roles should access user activity logs:

```
TEXT
Tenant Admin
Instance Admin
Security Auditor optional
```

Users should be allowed to request their own activity export if required by local policy, but they must not be able to modify it.

9.7 Activity Log UI

The Tenant Admin UI should include:

- filter by user
- filter by assessment
- filter by action type
- filter by date range
- filter by entity type
- search by finding/risk/question ID
- export activity log
- view before/after changes
- highlight high-risk actions

High-risk actions:

- finding deleted
- risk accepted
- score reduced
- evidence status changed from Missing to Reviewed
- report exported
- encrypted email sent
- user role changed
- MFA disabled

10. Audit Logging

Audit logging is mandatory and separate from detailed activity logging.

Audit logs record security and administration events.

Events:

- login success
- login failure
- MFA enabled
- MFA disabled
- password changed
- user invited
- user disabled
- role changed
- settings changed
- backup started
- backup failed
- encryption key rotated
- report sent externally

Audit log requirements:

- append-only design
- timestamps in UTC
- actor user ID
- source IP if available
- user agent if available
- affected entity type
- affected entity ID
- action description
- exportable audit log

11. Application Modules

11.1 Guided Assessment Workflow

Main workflow:

```
TEXT
Setup
→ Scope & Context
→ Guided Questions
→ Finding Review
→ Risk & Roadmap
→ Report
```

The user should always understand the next required step.

11.2 Customer / Client Management

Features:

- create customer
- edit customer profile
- define industry and regulatory context
- define critical systems
- define business criticality
- archive customer
- export customer project

11.3 Assessment Setup

Features:

- assessment type selection
- role / audience selection
- framework selection
- language selection
- target date
- assessment status

11.4 Scope & Context

Features:

- in-scope systems
- out-of-scope areas

- business-critical processes
- regulatory context
- assumptions
- limitations
- criticality level

11.5 Framework Library / Compliance Engine

The Framework Library should operate as a local compliance engine.

Capabilities:

- framework catalogue
- framework domains
- framework controls
- mappings between questions and controls
- evidence requirements
- framework coverage calculations
- readiness score
- gap analysis
- report mapping
- licensed framework import
- framework license status tracking

Supported Alpha frameworks and framework modes:

TEXT	
Framework	Delivery Mode
ISO/IEC 27001:2022	Audity readiness workflow + user-provided licensed content import
ISO/IEC 27002:2022	Audity readiness workflow + user-provided licensed content import
NIS2	Built-in legal requirement mapping where licensing permits
NIST Cybersecurity Framework 2.0	Built-in framework support
CIS Controls v8	License status must be reviewed before redistribution
MITRE ATT&CK	Built-in detection and SOC coverage mapping where licensing permits
HIPAA Security Rule	Built-in healthcare security requirement mapping where licensing permits
NSM Grunnprinsipper for IKT-sikkerhet	Built-in or referenced mapping depending on source license
Custom Framework	User-created or user-imported framework pack

11.5.1 Proprietary Framework and Standards Content Handling

Some frameworks and standards cannot be redistributed as full built-in content because the official standard text, tables, control catalogues, translations, and structured extracts may be protected by copyright or specific licensing terms.

This is especially relevant for proprietary standards such as:

- ISO/IEC 27001
- ISO/IEC 27002
- other ISO/IEC standards
- commercially licensed control catalogues
- third-party audit frameworks with restrictive redistribution terms

Audity must therefore distinguish between:

- | TEXT |
|---|
| 1. Audity-native assessment content |
| 2. Public or openly redistributable framework content |
| 3. User-provided licensed framework content |
| 4. Custom internal framework content |

Audity must not ship copyrighted standard text unless the project has explicit redistribution rights.

11.5.2 ISO 27001 Support Model

Audity must support ISO 27001 assessments without redistributing the official ISO standard text.

The correct product model is:

```
TEXT
Audity supports ISO 27001 readiness workflows.
Audity does not include official ISO/IEC 27001 standard text.
Audity does not include the official ISO/IEC 27001 control catalogue as redistributed copyrighted content.
Customers may import their own licensed ISO content if they have the right to use it.
```

Audity may include original, Audity-written ISO readiness content, such as:

- ISMS scope questions
- leadership and responsibility questions
- risk assessment questions
- risk treatment questions
- Statement of Applicability readiness questions
- policy and procedure review questions
- internal audit readiness questions
- management review readiness questions
- corrective action questions
- access control readiness questions
- identity management readiness questions
- backup and recovery readiness questions
- logging and monitoring readiness questions
- incident management readiness questions
- supplier security readiness questions
- vulnerability management readiness questions

Audity must avoid including:

- official ISO standard text
- full ISO tables
- official ISO control descriptions
- copied ISO implementation guidance
- copied ISO translations
- reconstructed proprietary control catalogues that effectively reproduce the standard

Recommended UI label:

```
TEXT
ISO/IEC 27001:2022
Status: Supported through Audity readiness workflow and user-provided licensed content.
Official ISO standard text is not included.
```

Recommended report wording:

```
TEXT
This ISO 27001 readiness assessment is based on Audity assessment questions and framework mappings. Official ISO st
```

If the tenant imports licensed ISO content, the report should state:

TEXT

ISO framework content used in this assessment was provided by the tenant and used within this self-hosted Audity in

11.5.3 Bring Your Own Framework Content

Audity should provide a “Bring Your Own Framework Content” feature.

Purpose:

TEXT

Allow users to import framework or standard content that their organization is licensed to use.

Supported import formats:

- CSV
- JSON
- YAML
- XLSX planned

Example import fields:

TEXT

framework_name
framework_version
control_id
control_title
domain
requirement_summary
evidence_examples
mapping_tags
license_source

Before importing licensed framework content, Audity must show a confirmation prompt:

TEXT

You are importing licensed or third-party framework content. Confirm that your organization has the right to use th

The import process must store metadata:

TEXT

imported_by
imported_at
source_filename
source_type
license_confirmed
license_note
framework_pack_id
distributed_by_audity = false

11.5.4 Framework License Metadata

The framework data model must include licensing and distribution metadata.

Recommended fields:

```
TEXT
source_type:
- audity_native
- public_source
- user_imported
- licensed_third_party
- custom_internal

license_status:
- built_in_allowed
- user_license_required
- redistribution_not_allowed
- license_review_required
- unknown

distributed_by_audity: true / false
imported_by_user: true / false
license_confirmed: true / false
```

Example for ISO 27001:

```
TEXT
name: ISO/IEC 27001
version: 2022
source_type: licensed_third_party
license_status: user_license_required
distributed_by_audity: false
imported_by_user: true
license_confirmed: tenant-controlled
```

Example for Audity-native content:

```
TEXT
name: Audity ISO 27001 Readiness Questions
version: internal
source_type: audity_native
license_status: built_in_allowed
distributed_by_audity: true
imported_by_user: false
license_confirmed: not_required
```

11.5.5 Framework Library UI Requirements

The Framework Library UI must clearly show the content status for each framework.

Each framework card should display:

- framework name
- version
- type
- source type
- license status
- built-in content availability
- import status
- whether official standard text is included
- whether user-provided licensed content is available

Example UI states:

TEXT
Built-in
This framework is included with Audity.

TEXT
Readiness Workflow Only
Audity includes original assessment questions and mappings, but not official standard text.

TEXT
User License Required
Import licensed framework content owned by your organization.

TEXT
License Review Required
Do not redistribute this framework until the license terms have been reviewed.

11.5.6 Open and Public Framework Strategy

Audity should prioritize open, public, or redistributable frameworks for built-in content.

Potentially suitable built-in candidates include:

- NIST Cybersecurity Framework 2.0
- NIST SP 800-61
- NIST SP 800-53 where redistribution terms permit
- selected public legal requirement mappings
- Audity-native control library
- user-created custom frameworks

Before including any third-party framework as built-in content, the project must review:

- copyright status
- redistribution rights
- modification rights
- attribution requirements
- commercial-use restrictions
- database extraction restrictions
- translation restrictions

11.5.7 Required Disclaimer

Audity must include the following disclaimer in the Framework Library and documentation:

TEXT
Audity does not redistribute copyrighted standards content unless explicitly licensed. For proprietary standards su

Users are responsible for ensuring that imported framework content is properly licensed for their organization and

Audity should use readiness terminology instead of final compliance claims.

Use:

```
TEXT
Readiness
Coverage
Evidence completeness
Potential gap
Assessment result
Framework mapping
```

Avoid unsupported claims:

```
TEXT
Certified
Fully compliant
Guaranteed compliance
Audit-proof
Official ISO audit
```

11.6 Guided Questions

Features:

- domain-based control questions
- score 0–5
- answer state
- evidence status
- confidence level
- notes
- framework mapping panel
- evidence expectations

Scoring model:

```
TEXT
0 = Not existing
1 = Ad hoc
2 = Partially documented
3 = Defined and implemented
4 = Measured and regularly reviewed
5 = Optimized
```

11.7 Finding Review

Features:

- suggested findings
- source explanation
- framework mappings
- accept finding
- edit finding
- dismiss finding
- mark as accepted risk

11.8 Risk Register

Features:

- create risk from finding
- likelihood score
- impact score

- calculated rating
- treatment option
- risk owner
- treatment plan
- due date
- status

Risk formula:

```
TEXT
Risk Score = Likelihood × Impact
```

Rating model:

```
TEXT
1-4      = Low
5-9      = Medium
10-16    = High
17-25    = Critical
```

11.9 Roadmap Builder

Roadmap phases:

- 0–30 days
- 31–90 days
- 3–6 months
- 6–12 months

Features:

- roadmap action generation
- manual adjustment
- owner assignment
- due date
- effort estimate
- status tracking

12. Branded Report Builder

12.1 Purpose

Audity must support professional report exports with customer-specific or tenant-specific branding.

Reports should be suitable for:

- executive stakeholders
- technical teams
- boards
- auditors
- customers
- internal security teams

12.2 Admin-Controlled Branding

Only Tenant Admins or users with `branding.manage` permission may configure branding.

Standard users must not be able to change:

- company logo
- report color scheme
- report template style
- footer branding
- default contact information layout

Admin-controlled branding settings:

- company logo
- primary color
- secondary color
- accent color
- report cover style
- header style
- footer style
- typography preset
- confidentiality label
- watermark option
- page numbering style

12.3 Modular Report Design System

The report designer should use a modular building-block model.

Available report blocks:

- cover page
- document control
- executive summary
- scope and methodology
- organization context
- maturity overview
- framework readiness
- top risks
- detailed findings
- risk register
- roadmap
- evidence overview
- assumptions and limitations
- appendix
- contact page

Admins can define which blocks are included in each report type.

Example report templates:

```
TEXT
Executive Report
Technical Report
Full Assessment Report
Board Summary
Remediation Roadmap
```

12.4 Required Author Information

Every exported report must include the creator's name and contact details.

Required fields:

- report creator name
- role / title
- email address
- phone number optional
- organization
- report creation date

This information must be shown in:

- document control section
- footer or contact page
- final report metadata

12.5 Report Export Formats

Minimum:

```
TEXT
PDF
```

Planned:

```
TEXT
DOCX
XLSX risk register
CSV findings export
JSON project export
```

12.6 Report Integrity

Each exported report should include:

- report generation timestamp
- report version
- assessment ID
- creator ID
- optional checksum
- optional export audit log reference

Exporting a report must create an audit log and user activity log event.

13. Secure Email Module

13.1 Purpose

Audity should include an email module for securely sending audit reports and assessment packages.

The module must not send unencrypted sensitive audit reports as normal email attachments.

13.2 Supported Sending Modes

Minimum secure model:

```
TEXT
Send encrypted report package as attachment
Send password / decryption secret through a separate channel
```

Supported future models:

- PGP encrypted attachment
- S/MIME encrypted email
- password-protected encrypted ZIP / AES package
- secure download link with short expiration
- recipient identity verification

13.3 Encrypted Report Package

Audity should generate an encrypted package before sending.

Example:

```
TEXT
Assessment_Report_Artemis_2026.auditysecure
```

Package content:

- PDF report
- optional risk register
- optional evidence summary
- metadata file
- checksum

Encryption requirements:

- strong symmetric encryption
- unique encryption key per package
- password-based encryption using strong KDF if password mode is used
- no plaintext report left in temporary directories after sending

13.4 Email Configuration

Self-hosted admins should configure SMTP settings.

Settings:

- SMTP host
- SMTP port
- TLS mode
- SMTP username
- SMTP password / app password
- sender address
- sender display name
- allowed sender domain optional

SMTP secrets must be encrypted at rest.

13.5 Recipient Controls

Before sending, Audity must require:

- recipient email address
- recipient name optional
- report package selection
- confirmation dialog
- encryption method
- optional message text

High-risk warning:

TEXT

You are about to send sensitive audit material externally. Confirm recipient, encryption method, and delivery channel.

13.6 Email Audit Trail

Every email action must be logged.

Log fields:

- sender user ID
- recipient email
- report ID
- assessment ID
- timestamp
- encryption method
- delivery status
- attachment/package ID
- SMTP result

Do not log encryption passwords or plaintext package keys.

13.7 Admin Controls

Tenant Admins should configure:

- whether email sending is enabled
- allowed sender domains
- maximum attachment size
- required encryption method
- whether external recipients are allowed
- whether approval is required before sending

14. Import / Export

Features:

- export project backup
- import project backup
- export report
- export risk register
- export audit log
- export user activity log

Project export format:

TEXT

*.cisoassess

Future format:

TEXT

.cisoassess = encrypted ZIP-like project container

15. Database Design Overview

Recommended database:

TEXT
PostgreSQL

Core tables:

TEXT
users
roles
permissions
role_permissions
sessions
mfa_settings
customers
assessments
assessment_frameworks
frameworks
framework_domains
framework_controls
assessment_questions
control_mappings
control_answers
findings
risks
roadmap_items
evidence_items
reports
report_templates
report_branding
email_settings
email_delivery_log
audit_logs
user_activity_logs
settings
backup_jobs

16. Docker Compose Concept

Example structure:

```
YAML
services:
  audity-web:
    image: audity/web:latest
    restart: unless-stopped
    depends_on:
      - audity-api

  audity-api:
    image: audity/api:latest
    restart: unless-stopped
    env_file:
      - .env
    depends_on:
      - audity-db
      - audity-redis

  audity-worker:
    image: audity/worker:latest
    restart: unless-stopped
    env_file:
      - .env
    depends_on:
      - audity-db
      - audity-redis

  audity-db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_DB: audity
      POSTGRES_USER: audity
      POSTGRES_PASSWORD: change-me
    volumes:
      - audity_postgres:/var/lib/postgresql/data

  audity-redis:
    image: redis:7-alpine
    restart: unless-stopped
    volumes:
      - audity_redis:/data

  audity-storage:
    image: minio/minio
    restart: unless-stopped
    command: server /data --console-address ":9001"
    volumes:
      - audity_storage:/data

volumes:
  audity_postgres:
  audity_redis:
  audity_storage:
```

17. Environment Variables

Example `.env`:

```
ENV
AUDITY_ENV=production
AUDITY_PUBLIC_URL=https://audity.example.com
AUDITY_APP_SECRET=replace-with-secure-random-secret
AUDITY_ENCRYPTION_KEY=replace-with-base64-encoded-32-byte-key
AUDITY_DATABASE_URL=postgres://audity:change-me@audity-db:5432/audity
AUDITY_REDIS_URL=redis://audity-redis:6379
AUDITY_STORAGE_PROVIDER=minio
AUDITY_STORAGE_ENDPOINT=http://audity-storage:9000
AUDITY_STORAGE_BUCKET=audity-evidence
AUDITY_STORAGE_ACCESS_KEY=replace-me
AUDITY_STORAGE_SECRET_KEY=replace-me
AUDITY_REQUIRE_MFA=false
AUDITY_PUBLIC_REGISTRATION=false
AUDITY_SMTP_ENABLED=false
AUDITY_SMTP_HOST=smtp.example.com
AUDITY_SMTP_PORT=587
AUDITY_SMTP_TLS=true
AUDITY_SMTP_USER=replace-me
AUDITY_SMTP_PASSWORD=replace-me
AUDITY_LOG_LEVEL=info
```

18. Backup and Recovery

Backup requirements:

- PostgreSQL backup
- evidence storage backup
- report package backup
- configuration backup
- encryption key backup strategy
- audit log backup
- user activity log backup
- restore test procedure

Recommended backup approach:

```
TEXT
Daily database backups
Daily evidence storage backups
Weekly full backup archive
Retention policy configurable by operator
Restore test documented
```

Important:

```
TEXT
If encryption keys are lost, encrypted customer data may not be recoverable.
```

19. Logging and Monitoring

Self-hosted operators should monitor:

- container health
- database availability
- disk usage

- backup status
- failed login attempts
- API errors
- worker failures
- report generation errors
- email delivery errors
- storage errors
- unusual activity log volume

Recommended tools:

- Uptime Kuma
- Prometheus
- Grafana
- Loki
- Docker health checks

20. Security Hardening Recommendations

Recommended host hardening:

- run Linux with regular security updates
- enable firewall
- expose only required ports
- use HTTPS with valid certificates
- disable password SSH login
- use SSH keys
- restrict admin access
- separate production and test environments
- run containers with least privilege
- avoid privileged containers
- set resource limits
- rotate secrets
- backup before upgrades

Recommended container hardening:

- non-root container users
- read-only filesystem where possible
- minimal base images
- pinned image versions
- vulnerability scanning
- no secrets baked into images
- Docker network isolation
- health checks

21. Future AI Architecture

AI should be optional and disabled by default.

Future AI features may include:

- finding text improvement

- executive summary drafting
- interview note summarization
- report language refinement
- evidence gap explanation

Security requirements for AI:

- tenant-local AI runtime
- no shared embeddings across customers
- no shared vector database across customers
- no training on customer data
- AI actions logged
- AI output marked as draft
- user must review and approve AI output

Possible future stack:

```
TEXT
audity-ai
vector-db
ollama/local-llm
tenant-specific document index
```

22. Open Source Positioning

The self-hosted Docker edition can be released as a free open-source version.

Recommended model:

```
TEXT
Community Edition:
- self-hosted Docker deployment
- local framework engine
- guided assessments
- findings
- risks
- roadmap
- reports
- basic user management
- MFA
- audit logs
- user activity logs
- report branding
- encrypted report package export
```

Possible future paid/commercial add-ons:

```
TEXT
Managed hosting
Enterprise support
SSO/SAML
SCIM
advanced RBAC
custom branding
advanced reporting
secure email approval workflows
private AI add-on
priority security updates
long-term support releases
```

23. Implementation Phases

Phase 1 — Docker Foundation

- split current web app into frontend/backend architecture
- add PostgreSQL
- add Docker Compose
- add environment-based configuration
- add health checks
- add initial admin setup

Phase 2 — Authentication and Security

- user login
- password hashing
- MFA TOTP
- session security
- RBAC
- audit logging
- user activity logging

Phase 3 — Data Persistence

- customers
- assessments
- framework engine
- answers
- findings
- risks
- roadmap
- reports
- evidence metadata

Phase 4 — Traceability and Integrity

- immutable activity log
- before/after change tracking
- admin activity log UI
- tamper-evident event hashing
- exportable user activity logs

Phase 5 — Reporting and Branding

- branded report templates
- admin-controlled logo and color scheme
- modular report builder
- required creator contact information
- PDF export

Phase 6 — Secure Email Module

- SMTP settings
- encrypted report package generation

- secure send workflow
- email delivery audit trail
- admin email controls

Phase 7 — Evidence and Reporting Hardening

- file uploads
- secure evidence access
- report generation
- project export/import
- encrypted `.cisoassess` package

Phase 8 — Platform Hardening

- encryption at rest
- field-level encryption
- backup/restore process
- container hardening
- security headers
- rate limiting
- dependency scanning

Phase 9 — Optional AI

- tenant-local AI runtime
- AI job queue
- AI audit logging
- tenant-specific vector store

24. Alpha Limitations

The first Docker edition should clearly state:

- framework mappings are initial and not certification-grade
- Audity does not provide legal advice
- Audity does not certify compliance
- professional review is required before formal audit conclusions
- operators are responsible for securing their own self-hosted environment
- backup and key management are the operator's responsibility
- user activity logs may contain sensitive behavioral and assessment data and must be protected accordingly
- proprietary standards such as ISO/IEC standards are not redistributed unless explicit licensing permits it
- ISO readiness workflows are not a substitute for holding the official standard or using qualified audit professionals

25. Final Target State

Audity Self-Hosted Docker Edition should provide:

- a guided security assessment workflow
- local/self-hosted deployment
- strong authentication
- MFA via authenticator apps

- password hashing with Argon2id
- encrypted sensitive customer data
- RBAC
- audit logging
- immutable user activity logging
- framework compliance engine
- proprietary framework import support
- ISO 27001 readiness support without redistributing official ISO content
- evidence management
- findings management
- risk register
- remediation roadmap
- branded report generation
- admin-controlled report logo and color scheme
- creator contact details in every report
- secure encrypted email delivery module
- import/export
- Docker-based deployment
- optional future tenant-local AI

The end goal is a practical, secure, self-hosted audit and GRC assessment platform that helps professionals get from assessment scope to report faster, without forcing sensitive customer data into an external SaaS service.